



Digital Design

Chapter 6: Optimizations and Tradeoffs

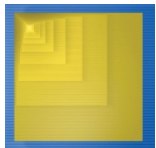
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

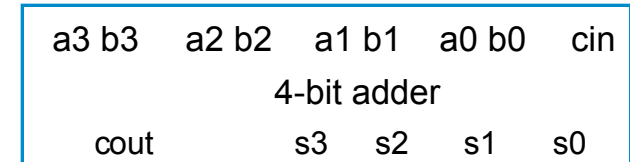
Datapath Component Tradeoffs

- Can make some components faster (but bigger), or smaller (but slower), than the straightforward components we built in Ch 4
- We' ll build
 - A faster (but bigger) adder than the carry-ripple adder
 - A smaller (but slower) multiplier than the array-based multiplier
- Could also do for the other Ch 4 components



Faster Adder

- Built carry-ripple adder in Ch 4
 - Similar to adding by hand, column by column
 - Con: Slow
 - Output is not correct until the carries have rippled to the left
 - 4-bit carry-ripple adder has $4 \times 2 = 8$ gate delays
 - Pro: Small
 - 4-bit carry-ripple adder has just $4 \times 5 = 20$ gates

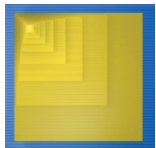
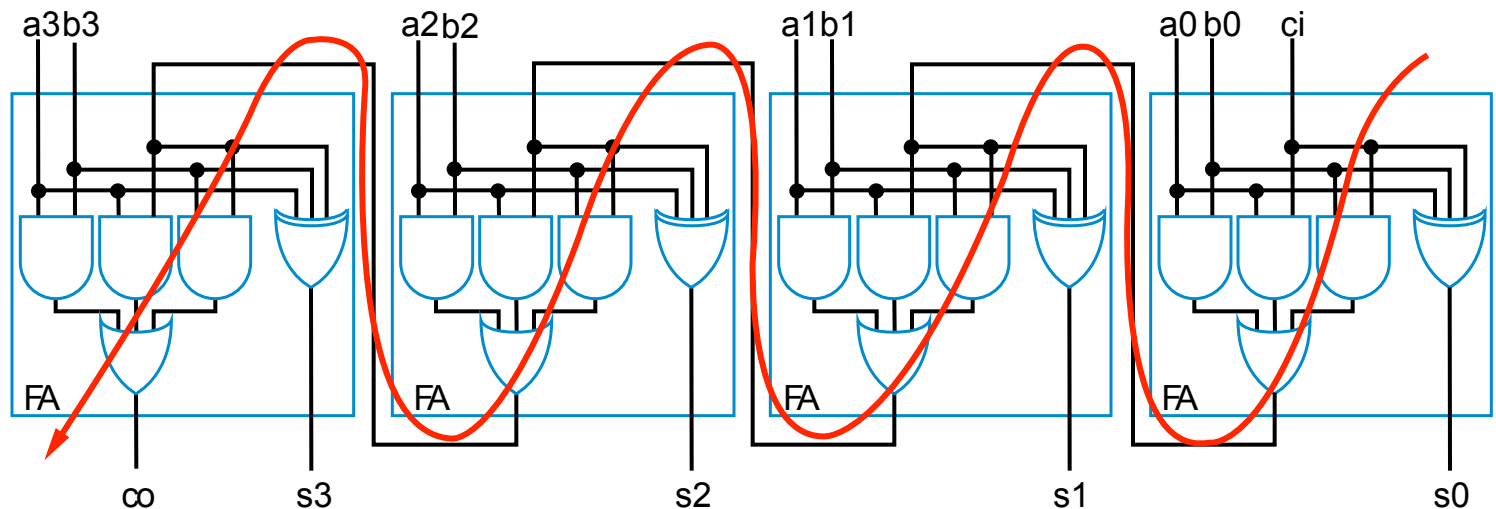


carries: **c3** **c2** **c1** cin

B: b3 b2 b1 b0

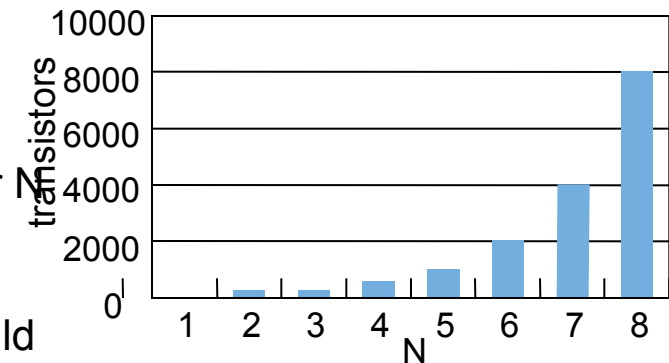
A: + a3 a2 a1 a0

cout s3 s2 s1 s0

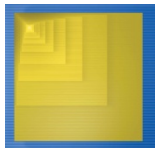
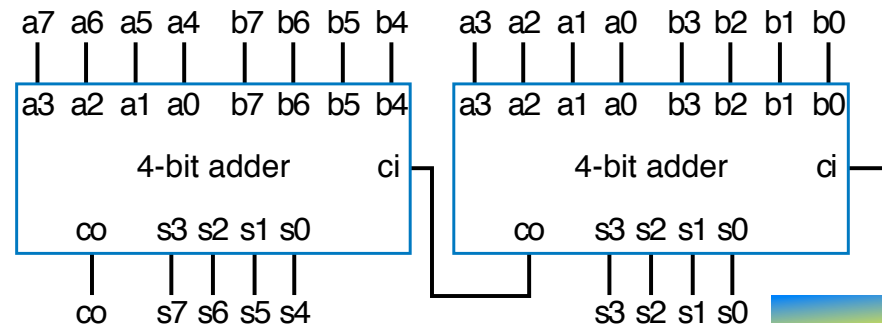


Building a Faster Adder

- Carry-ripple is slow but small
 - 8-bit: $8 \times 2 = 16$ gate delays, $8 \times 5 = 40$ gates
 - 16-bit: $16 \times 2 = 32$ gate delays, $16 \times 5 = 80$ gates
 - 32-bit: $32 \times 2 = 64$ gate delays, $32 \times 5 = 160$ gates
- Two-level logic adder (2 gate delays)
 - OK for 4-bit adder: About 100 gates
 - 8-bit: 8,000 transistors / 16-bit: 2 M / 32-bit: 100 billion
 - N-bit two-level adder uses *absurd* number of gates for N much beyond 4
- Compromise
 - Build 4-bit adder using two-level logic, compose to build N-bit adder
 - 8-bit adder: $2 \times (2 \text{ gate delays}) = 4$ gate delays, $2 \times (100 \text{ gates}) = 200$ gates
 - 32-bit adder: $8 \times (2 \text{ gate delays}) = 16$ gate delays, $8 \times (100 \text{ gates}) = 800$ gates



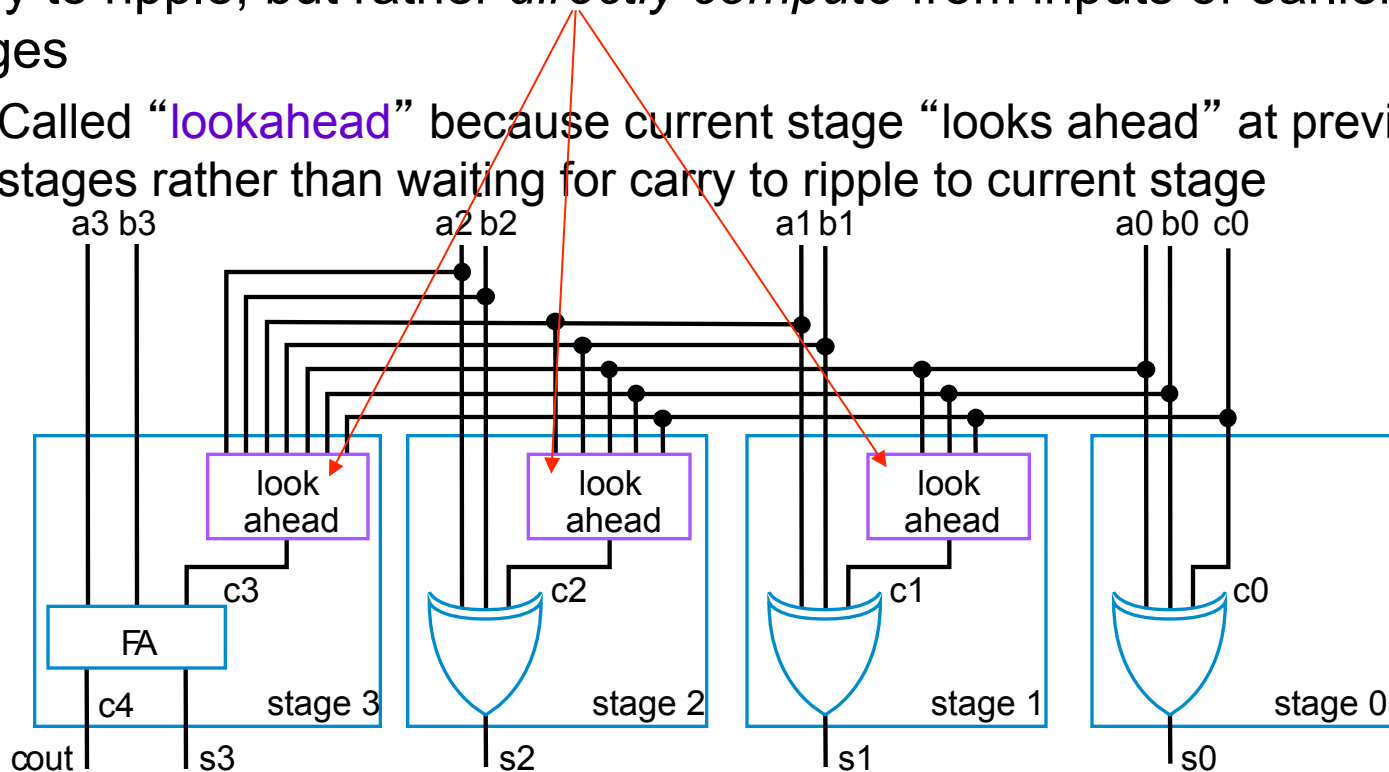
Can we do better?



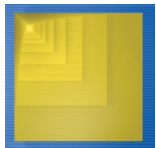
Faster Adder – (Bad) Attempt at “Lookahead”

- Idea

- Modify carry-ripple adder – For a stage's carry-in, don't wait for carry to ripple, but rather *directly compute* from inputs of earlier stages
- Called “**lookahead**” because current stage “looks ahead” at previous stages rather than waiting for carry to ripple to current stage

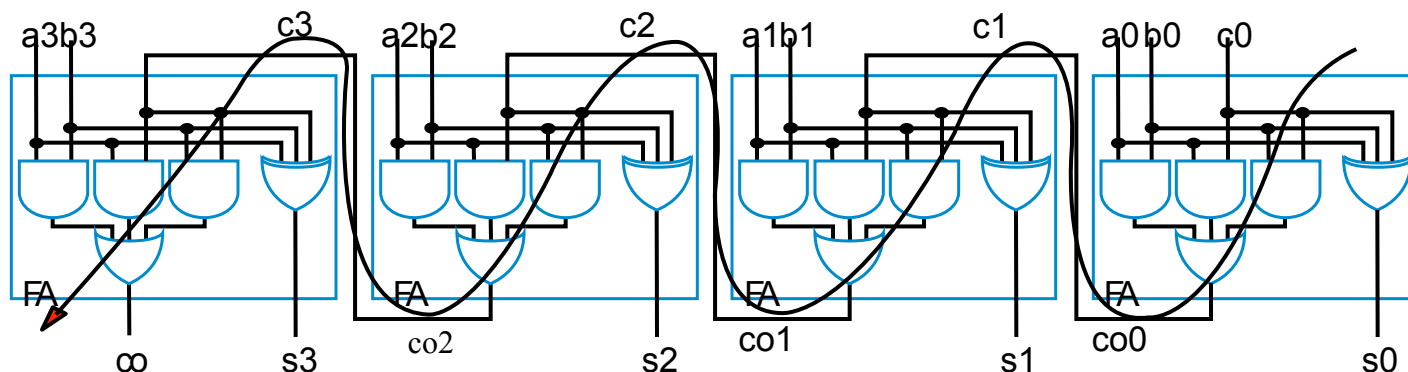


Notice – no rippling of carry



Faster Adder – (Naïve Inefficient) Attempt at “Lookahead”

- Want each stage's carry-in bit to be function of external inputs only (a' s, b' s, or $c0$)



- Full-adder: $s = a \text{ xor } b$ $c = bc + ac + ab$

$$c1 = co0 = b0c0 + a0c0 + a0b0$$

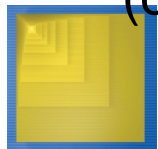
$$c2 = co1 = b1c1 + a1c1 + a1b1$$

$$c2 = b1(b0c0 + a0c0 + a0b0) + a1(b0c0 + a0c0 + a0b0) + a1b1$$

$$c2 = b1b0c0 + b1a0c0 + b1a0b0 + a1b0c0 + a1a0c0 + a1a0b0 + a1b1$$

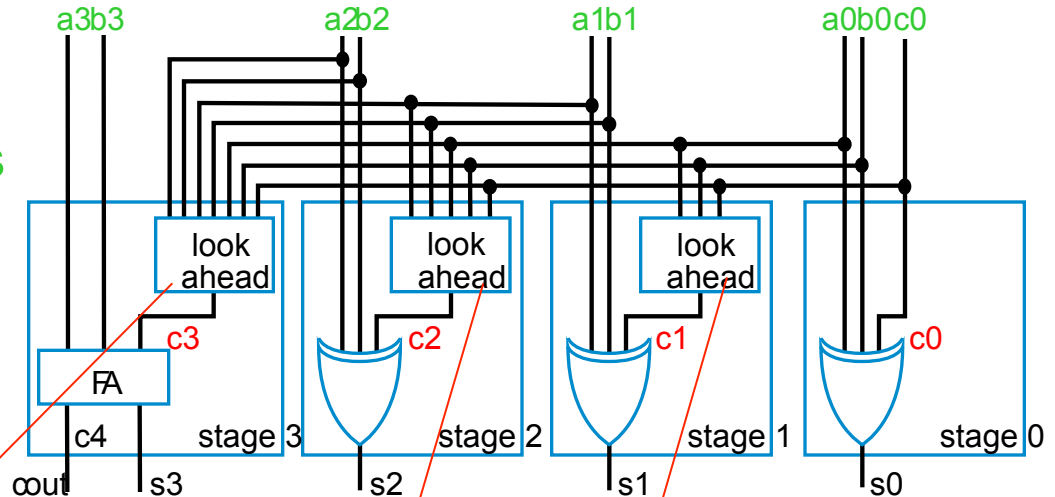
$$c3 = co2 = b2c2 + a2c2 + a2b2$$

(continue plugging in...)



Faster Adder – (Bad) Attempt at “Lookahead”

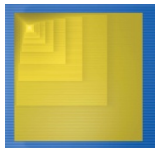
- **Carry** lookahead logic function of **external inputs**
 - No waiting for ripple
- Problem
 - Equations get too big
 - Not efficient
 - Need a better form of lookahead



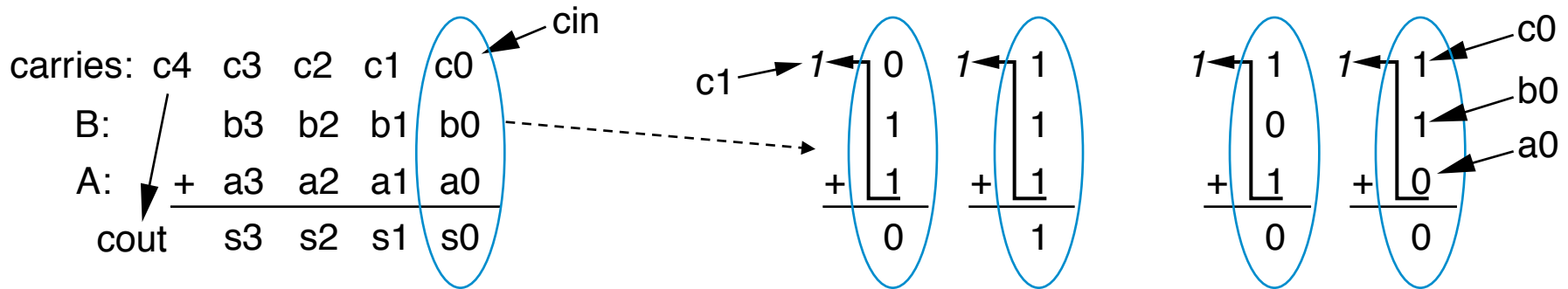
$$c_1 = b_0c_0 + a_0c_0 + a_0b_0$$

$$c_2 = b_1b_0c_0 + b_1a_0c_0 + b_1a_0b_0 + a_1b_0c_0 + a_1a_0c_0 + a_1a_0b_0 + a_1b_1$$

$$c_3 = b_2b_1b_0c_0 + b_2b_1a_0c_0 + b_2b_1a_0b_0 + b_2a_1b_0c_0 + b_2a_1a_0c_0 + b_2a_1a_0b_0 + b_2a_1b_1 + a_2b_1b_0c_0 + a_2b_1a_0c_0 + a_2b_1a_0b_0 + a_2a_1b_0c_0 + a_2a_1a_0c_0 + a_2a_1a_0b_0 + a_2a_1b_1 + a_2b_2$$



Efficient Lookahead



$$c_1 = a_0b_0 + (a_0 \text{ xor } b_0)c_0$$

$$c_2 = a_1b_1 + (a_1 \text{ xor } b_1)c_1$$

$$c_3 = a_2b_2 + (a_2 \text{ xor } b_2)c_2$$

$$c_4 = a_3b_3 + (a_3 \text{ xor } b_3)c_3$$

$$c_1 = G_0 + P_0c_0$$

$$c_2 = G_1 + P_1c_1$$

$$c_3 = G_2 + P_2c_2$$

$$c_4 = G_3 + P_3c_3$$

$$G_i = a_i b_i \text{ (generate)}$$

$$P_i = a_i \text{ XOR } b_i \text{ (propagate)}$$

$$c_1 = G_0 + P_0c_0 \text{ (Can prove that same as } b_0c_0 + a_0c_0 + a_0b_0 \text{)}$$

Why those names? When $a_0b_0=1$, we should generate a 1 for c_1 . When $a_0 \text{ XOR } b_0 = 1$, we should propagate the c_0 value as the value of c_1 , meaning c_1 should equal c_0 .



Efficient Lookahead

- Substituting as in the naïve scheme:

$$c1 = G0 + P0c0$$

$$c2 = G1 + P1c1$$

$$c2 = G1 + P1(G0 + P0c0)$$

$$c2 = G1 + P1G0 + P1P0c0$$

$$c3 = G2 + P2c2$$

$$c3 = G2 + P2(G1 + P1G0 + P1P0c0)$$

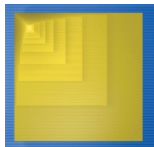
$$c3 = G2 + P2G1 + P2P1G0 + P2P1P0c0$$

$$c4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0c0$$

$G_i = a_i b_i$ (generate)

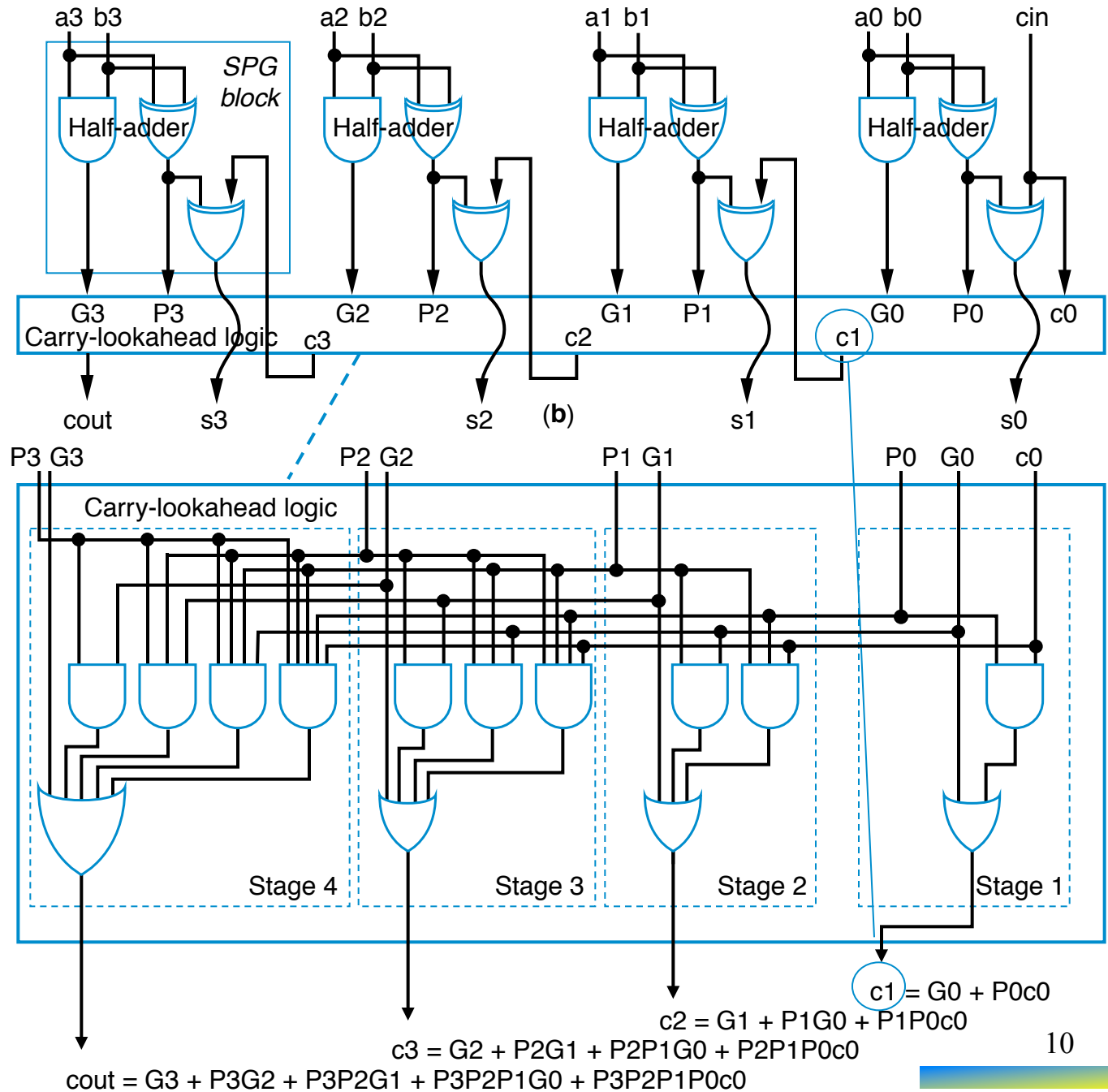
$P_i = a_i \text{ XOR } b_i$ (propagate)

G_i/P_i function of inputs only

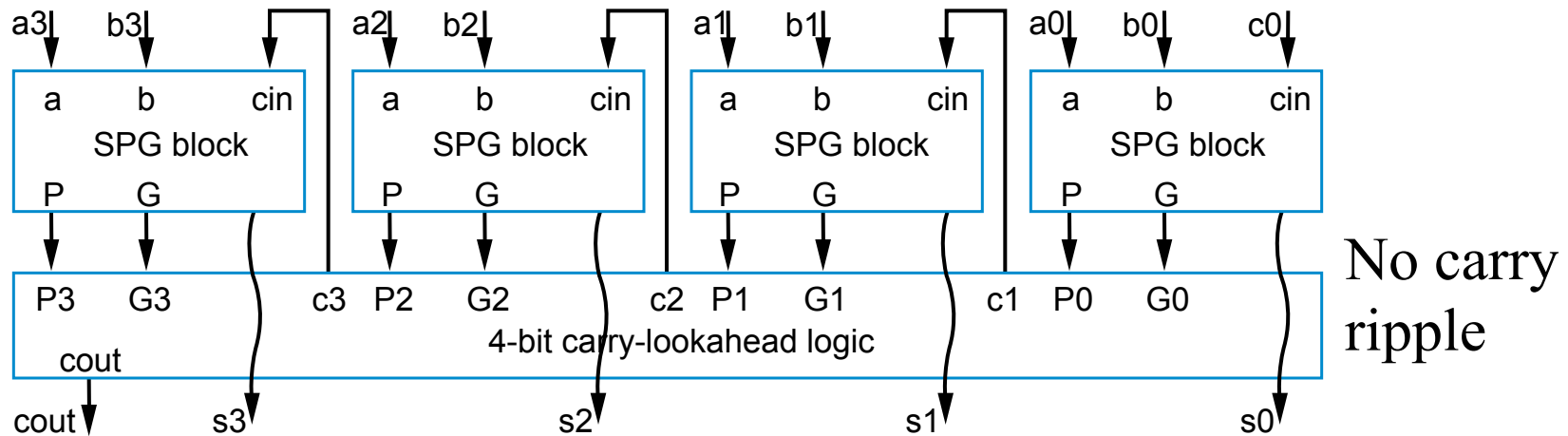


CLA

- Each stage:
 - HA for G and P
 - Another XOR for s
 - Call SPG block
- Create carry-lookahead logic from equations
- More efficient than naïve scheme, at expense of one extra gate delay



Carry-Lookahead Adder -- High-Level View

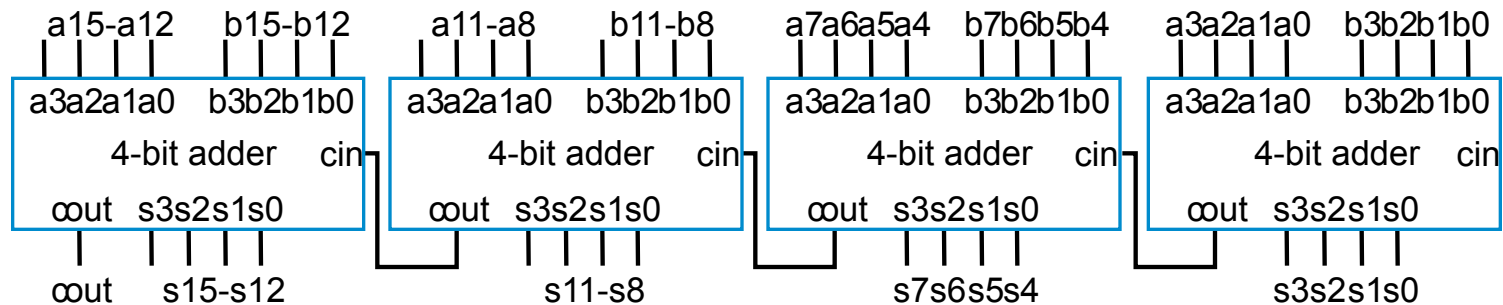
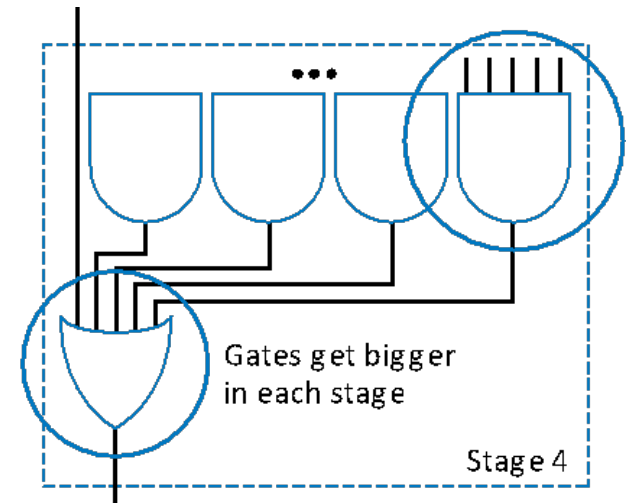


- Fast -- only **4 gate delays**
 - Each stage has SPG block with 2 gate levels
 - Carry-lookahead logic quickly computes the carry from the propagate and generate bits using 2 gate levels inside
- Reasonable number of gates -- 4-bit adder has only **26 gates (12 + 14)** (we should count gate inputs instead)
- 8-bit adder 64 gates, 4 gate delays
- 16 bit adder 200 gates 4 gate delays
- 32 bit adder 656 gates 4 gate delays
- 4-bit adder comparison (gate delays, gates)
 - Carry-ripple: (8, 20)
 - Two-level: (2, 500)
 - CLA: (4, 26)
 - o Nice compromise



Carry-Lookahead Adder – 32-bit?

- **Problem:** Gates get bigger in each stage
 - 4th stage has 5-input gates
 - 32nd stage would have 33-input gates
 - Too many inputs for one gate
 - Would require building from smaller gates, meaning more levels (slower), more gates (bigger)
- **One solution:** Connect 4-bit CLA adders in ripple manner
 - But slow (4 + 4 + 4 + 4 gate delays)

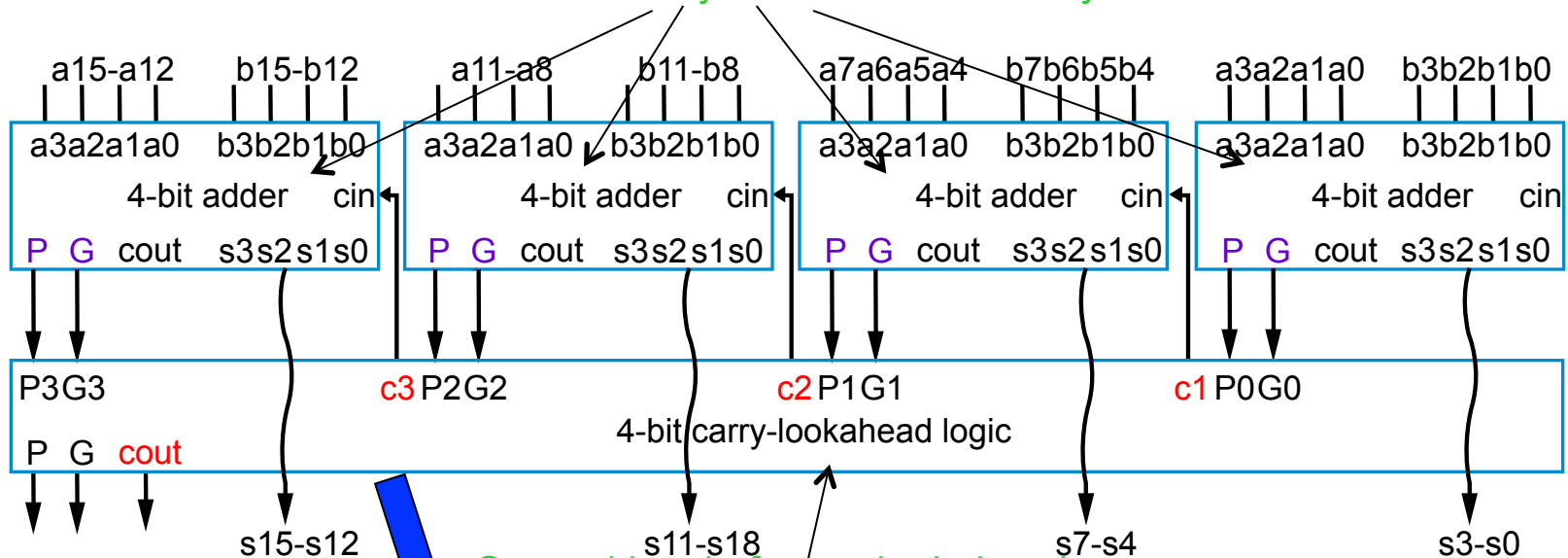


With carry ripple

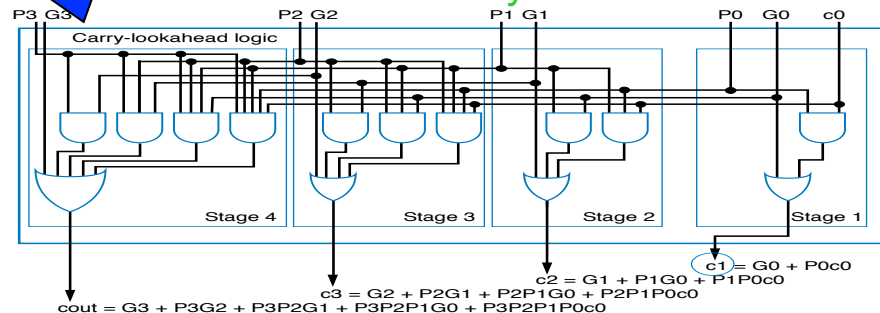
Hierarchical Carry-Lookahead Adders

- Better solution – Rather than rippling the carries, just *repeat* the carry-lookahead concept
 - Requires minor modification of 4-bit CLA adder to **output P and G**

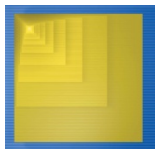
These use carry-lookahead internally



Second level of carry-lookahead

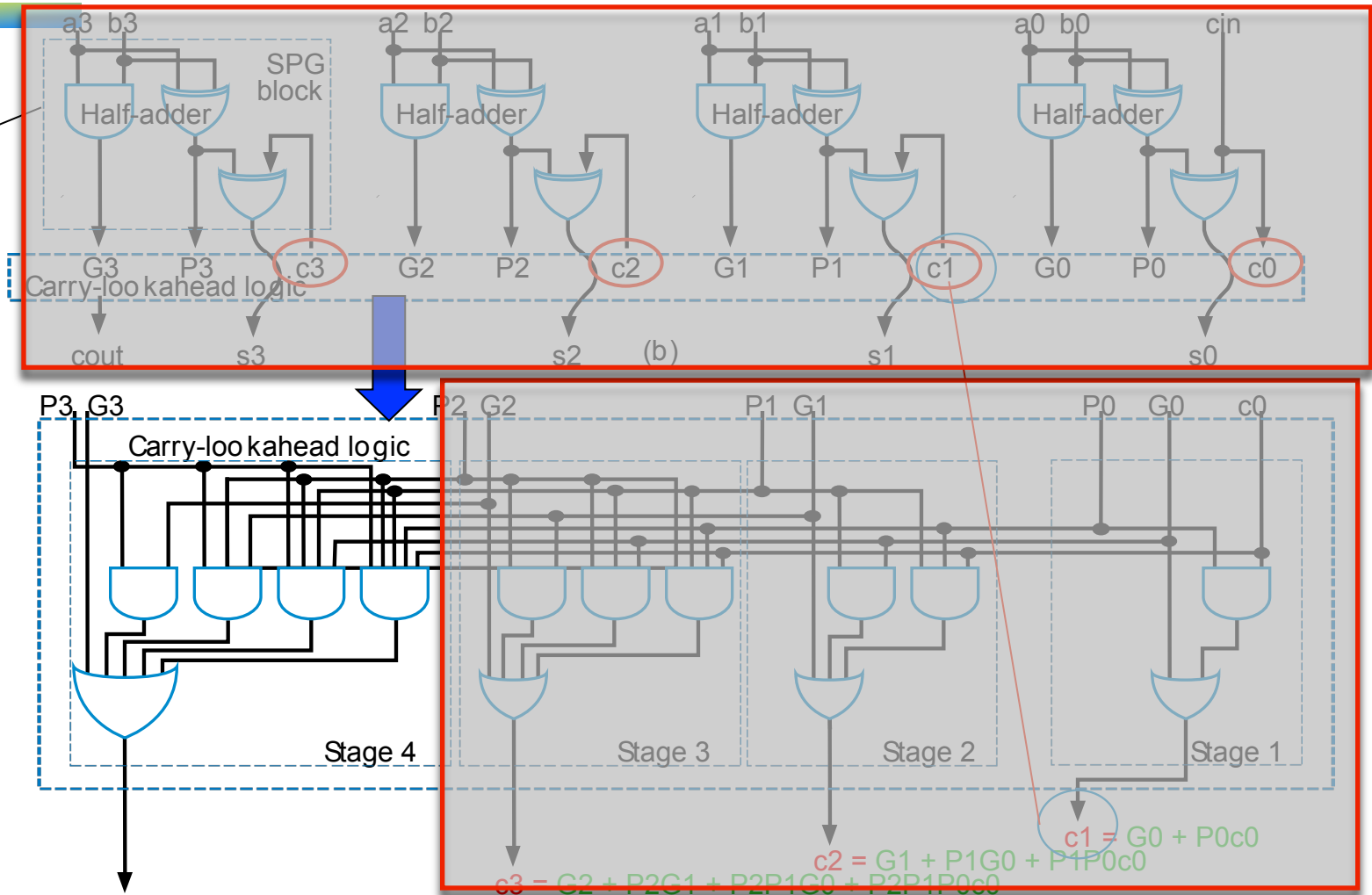


Same lookahead logic as inside the 4-bit adders



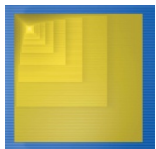
Lookahead

Call this sum/propagate/
generate (SPG) block



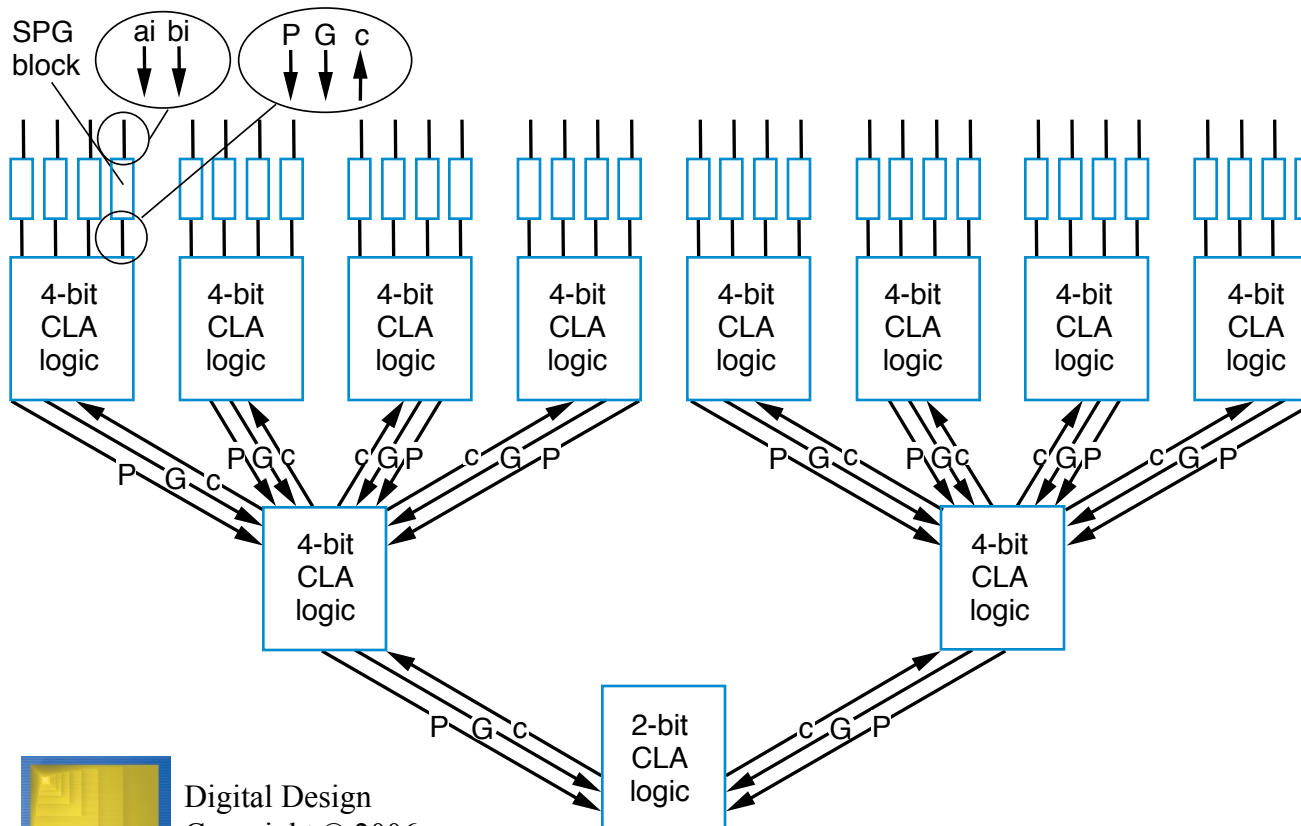
$$cout = c1 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0c0$$

- $G = G3 + P3G2 + P3P2G1 + P3P2P1G0$
- $P = P3P2P1P0$



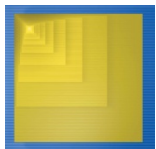
Hierarchical Carry-Lookahead Adders

- Hierarchical CLA concept can be applied for larger adders
- 32-bit hierarchical CLA:
 - Only about 8 gate delays (2 for SPG block, then 2 per CLA level)
 - Only about 14 gates in each 4-bit CLA logic block



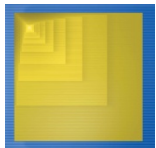
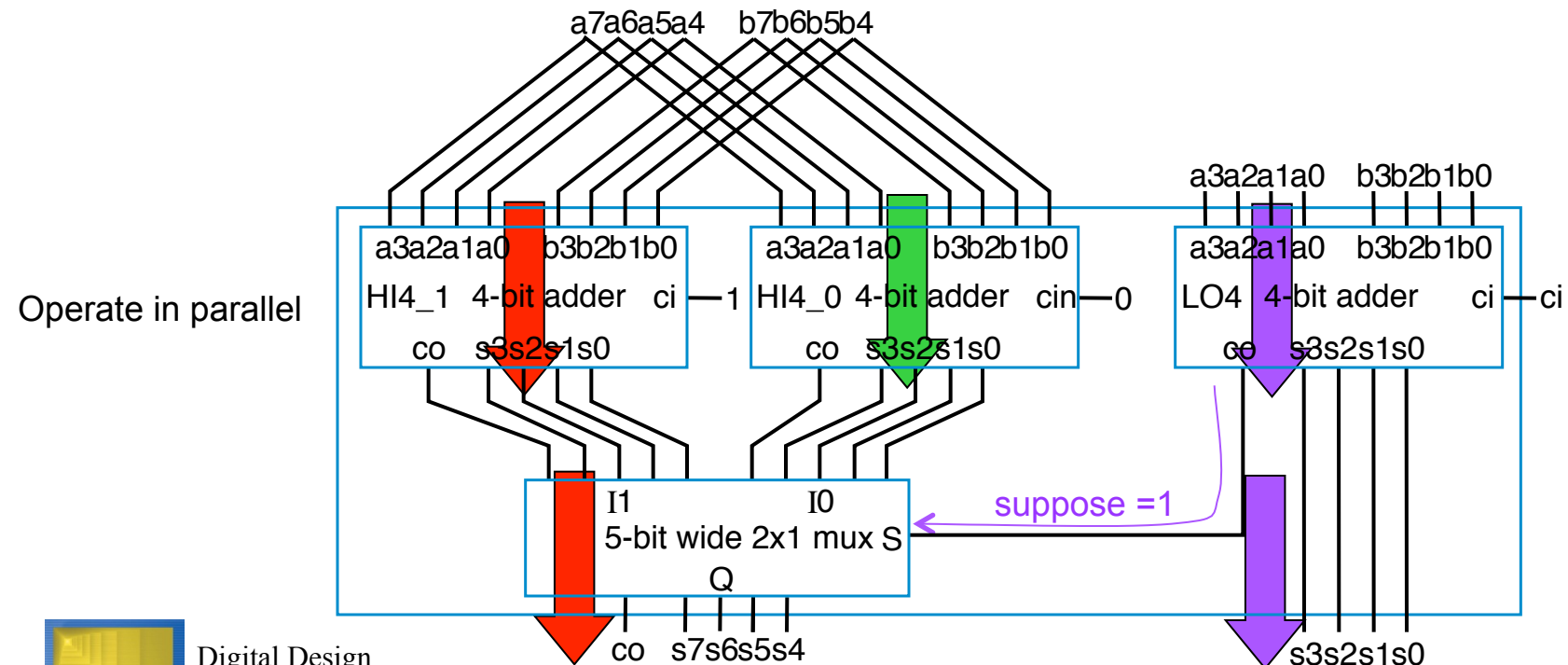
Q: How many gate delays for 64-bit hierarchical CLA, using 4-bit CLA logic?

A: 16 CLA-logic blocks in 1st level, 4 in 2nd, 1 in 3rd -- so still just 8 gate delays (2 for SPG, and 2+2+2 for CLA logic). *CLA is a very efficient method.*

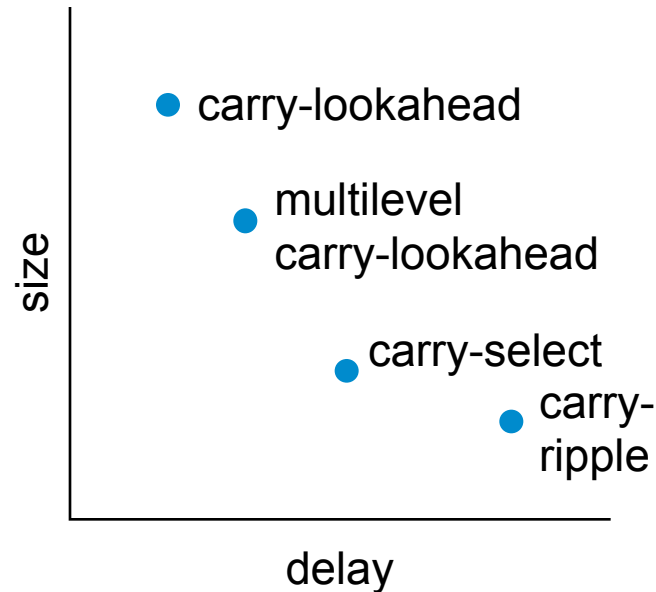


Carry Select Adder

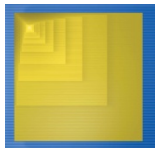
- Another way to compose adders
 - High-order stage – Compute result for carry in of **1** and of **0**
 - Select based on **carry-out** of low-order stage
 - Faster than pure rippling



Adder Tradeoffs



- Designer picks the adder that satisfies particular delay and size requirements
 - May use different adder types in different parts of same design
 - Faster adders on critical path, smaller adders on non-critical path

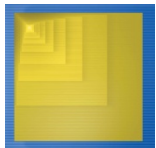
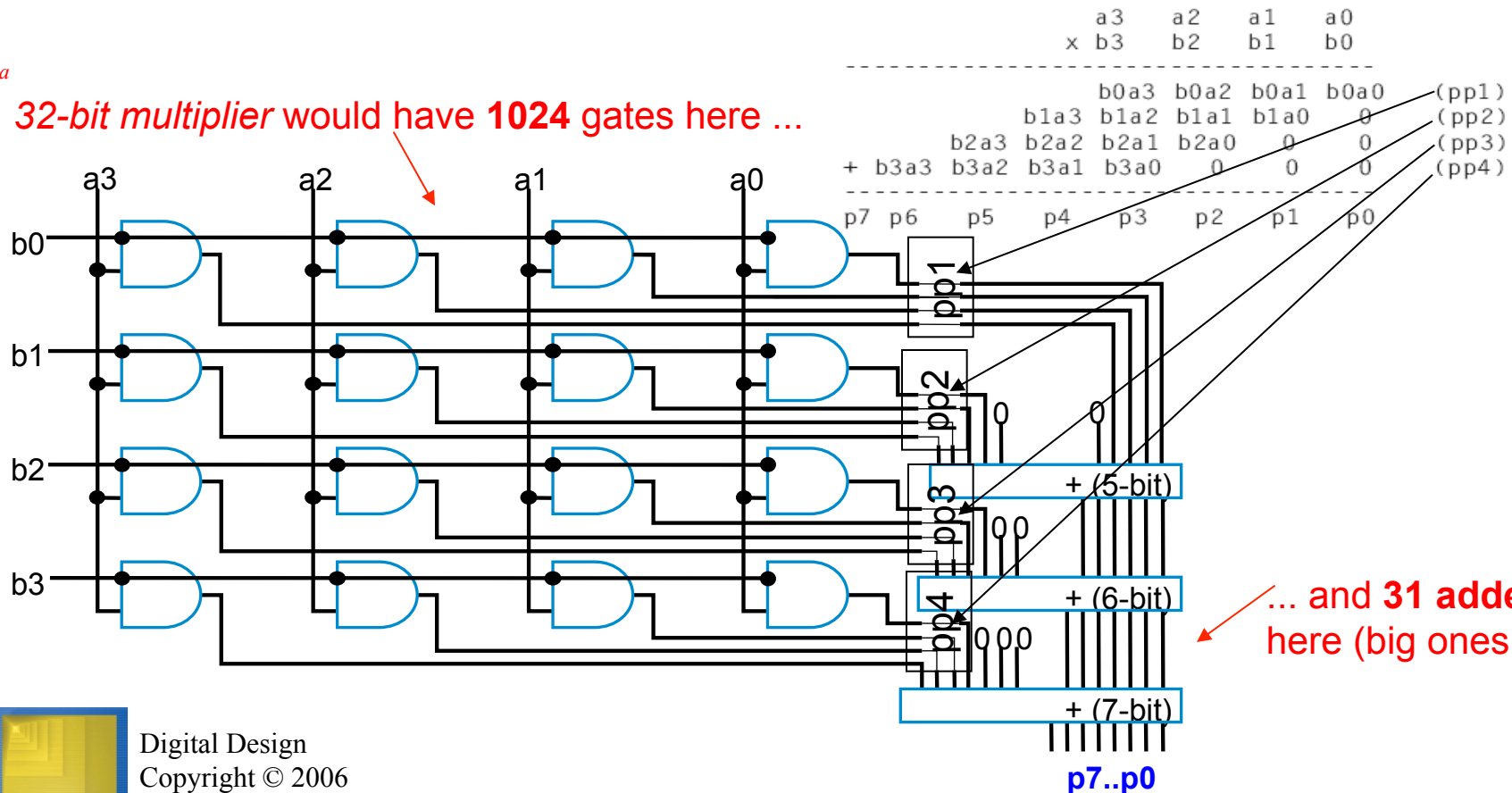


Smaller Multiplier

- Multiplier in Ch 4 was array style
 - Fast, reasonable size for 4-bit: $4 \times 4 = 16$ partial product AND terms, 3 adders
 - But big for 32-bit: $32 \times 32 = 1024$ AND terms, and 31 adders

a

32-bit multiplier would have 1024 gates here ...

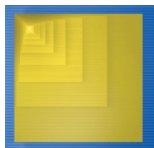


Smaller Multiplier -- Sequential (Add-and-Shift) Style

- Smaller multiplier: Basic idea
 - Don't compute all partial products simultaneously
 - Rather, compute one at a time (similar to by hand), maintain running sum

	Step 1	Step 2	Step 3	Step 4
	0 1 1 0	0 1 1 0	0 1 1 0	0 1 1 0
	X 0 0 1 1	X 0 0 1 1	0 0 1 1	0 0 1 1
(running sum)	0 0 0 0	0 0 1 1 0	0 1 0 0 1 0	0 0 1 0 0 1 0
(partial product)	+ 0 1 1 0	+ 0 1 1 0	+ 0 0 0 0	+ 0 0 0 0
(new running sum)	0 0 1 1 0	0 1 0 0 1 0	0 0 1 0 0 1 0	0 0 0 1 0 0 1 0

a



Smaller Multiplier -- Sequential (Add-and-Shift) Style

Design circuit that computes one partial product at a time, adds to running sum

- Note that **shifting** running sum right (relative to partial product) after each step ensures partial product added to correct running sum bits

Step 1	Step 2	Step 3	Step 4	
0 1 1 0	0 1 1 0	0 1 1 0	0 1 1 0	(running sum)
+ 0 0 1 1	+ 0 0 1 1	+ 0 0 1 1	+ 0 0 1 1	(partial product)
0 0 0 0	0 0 1 1 0	0 1 0 0 1 0	0 0 1 0 0 1 0	(new running sum)
+ 0 1 1 0	+ 0 1 1 0	+ 0 0 0 0	+ 0 0 0 0	
0 0 1 1 0	0 1 0 0 1 0	0 0 1 0 0 1 0	0 0 0 1 0 0 1 0	

